

4.2.2 Data Integrity and Constraints

Data integrity is essential to ensure that the information stored in the Smart Event Management and Ticket Booking System remains accurate, consistent, and reliable. The application uses MongoDB Atlas as the database and Mongoose as the Object Data Modeling (ODM) library to enforce data validation and maintain consistency.

Several constraints have been implemented to preserve data integrity:

Primary Data Constraints

- Each user is uniquely identified using MongoDB's ObjectId.
- Email addresses are unique to prevent duplicate user accounts.
- Required fields such as Name, Email, and Password must be provided during registration.

Validation Constraints

- Email addresses are validated before being stored.
- Password fields are mandatory and securely encrypted before storage.
- Empty or invalid input values are rejected by the application.

Database Integrity

- Duplicate user registrations are prevented using unique email constraints.
- Data consistency is maintained through Mongoose schema validation.
- Only authenticated users are allowed to access protected resources.

Data Storage

The application stores the following user information securely:

- User Name
- Email Address
- Encrypted Password
- User Role
- Account Creation Date

These constraints ensure that invalid, incomplete, or duplicate records are not stored in the database, thereby improving the reliability of the system.

4.4 Security Issues

Security plays a vital role in the Smart Event Management and Ticket Booking System. Various security mechanisms have been implemented to protect user data, prevent unauthorized access, and ensure secure communication between the frontend, backend, and database.

Authentication

- User authentication is implemented using **JSON Web Tokens (JWT)**.
- Every authenticated user receives a secure token after successful login.
- Protected routes can only be accessed with a valid JWT.

Password Security

- User passwords are encrypted using **bcrypt** before being stored in MongoDB Atlas.
- Plain-text passwords are never stored in the database.
- Password verification is performed using bcrypt comparison during login.

Authorization

- Role-Based Access Control (RBAC) is implemented.
- Separate authentication is provided for normal users and administrators.
- Administrative pages and APIs are accessible only to authorized administrators.

Database Security

- MongoDB Atlas provides secure cloud database hosting.
- Database credentials are stored as environment variables instead of hardcoding them into the application.
- Direct database access is restricted through authenticated connections.

API Security

- REST APIs validate incoming requests before processing.
- Unauthorized requests are rejected.
- Error handling middleware prevents exposure of sensitive server information.

Environment Security

Sensitive configuration values such as:

- MongoDB Connection URI
- JWT Secret Key
- Admin Credentials

- API Configuration

are stored securely using environment variables (`.env`) and are excluded from the public GitHub repository.

CORS Protection

Cross-Origin Resource Sharing (CORS) is configured to allow requests only from the authorized frontend domain, preventing unauthorized websites from accessing the backend APIs.

Deployment Security

The application is securely deployed using:

- **Vercel** (Frontend)
- **Railway** (Backend)
- **MongoDB Atlas** (Database)

All communication between the client and server is performed over **HTTPS**, ensuring secure transmission of data across the network.